

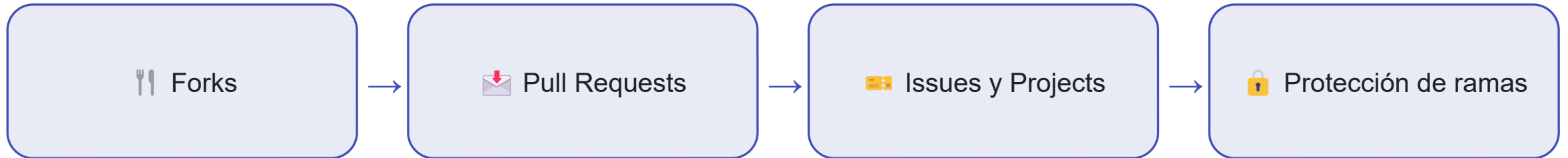
Colaboración y Gestión de Proyectos en GitHub

8. Colaboración y Gestión de Proyectos en GitHub

Git y GitHub no son solo una herramienta para guardar código: son la base sobre la que se organiza el trabajo en equipo en la mayoría de empresas de software.

En este apartado veremos cómo contribuir a proyectos ajenos y cómo gestionar el trabajo dentro de un equipo usando las herramientas que GitHub pone a disposición.

Contenidos



Forks (Bifurcaciones)

Un Fork es una copia de un repositorio ajeno que GitHub crea dentro de tu propia cuenta. La diferencia respecto a un clon: el fork vive en el servidor, bajo tu usuario, y tú tienes control total sobre él.

El caso típico: encuentras un proyecto que te interesa mejorar, pero no eres colaborador y no puedes hacer push directamente. Pulsas el botón «Fork» y GitHub copia todo el repositorio bajo tu cuenta en cuestión de segundos.

A partir de ahí puedes clonar tu copia, crear ramas, modificar lo que quieras y hacer push sin afectar en absoluto al proyecto original. GitHub mantiene la referencia al origen, lo que facilita proponer tus cambios de vuelta al autor mediante una Pull Request.

¿Fork o clone directo? Clona si ya eres colaborador con permisos de escritura. Haz un fork cuando quieras contribuir a un proyecto ajeno: tu fork actúa como puerta de entrada para proponer cambios mediante Pull Request.

Forks — Botón Fork en GitHub

Botón Fork en la esquina superior derecha de un repositorio ajeno. Al pulsarlo, GitHub crea la copia bajo tu cuenta en segundos.

The screenshot shows a GitHub repository page for 'mi_proyecto' by user 'aitorventura'. The repository is public and has 0 forks, 0 stars, and 0 watches. The main branch is 'main', and there are 2 branches and 0 tags. The repository contains a file named 'primer_doc.txt' which was changed yesterday. The repository has 2 commits. The right sidebar shows the 'About' section with no description, website, or topics provided, and the 'Releases' section with no releases published.

mi_proyecto Public

Watch 0 Fork 0 Star 0

main 2 Branches 0 Tags

Go to file t Add file <> Code

aitorventura Cambios 9e539b6 · yesterday 2 Commits

primer_doc.txt Cambios yesterday

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Forks — Pantalla de creación

Pantalla de creación del fork: se elige bajo qué cuenta o organización se crea la copia.

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *

 aitorventuraprofe ▾

Repository name *

/ mi_proyecto

✓ mi_proyecto is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

0 / 350 characters

☒ Copy the `main` branch only

Contribute back to aitorventura/mi_proyecto by adding your own branch. [Learn more.](#)

 You are creating a fork in your personal account.

Create fork

Sincronización del Fork

Con el tiempo, el repositorio original seguirá recibiendo commits. GitHub ofrece el botón Sync fork en la página principal de tu fork para traer esos cambios sin tener que hacerlo a mano.

¿Qué es upstream?

upstream es el repositorio original del que proviene tu fork. Tu copia es origin; el proyecto del que la hiciste es upstream. Al sincronizar, traes los cambios de upstream a tu origin.

Conflictos al sincronizar

Si el repositorio original y el tuyo han avanzado por separado, la sincronización puede generar conflictos que tendrás que resolver igual que con un git pull con cambios divergentes. Conviene sincronizar antes de empezar a trabajar en una nueva funcionalidad, no después.

Pull Requests (PRs)

Una Pull Request es una propuesta formal para integrar los commits de una rama en otra. Le estás diciendo al equipo: «He hecho estos cambios en esta rama; ¿los revisáis y los integráis en main?»

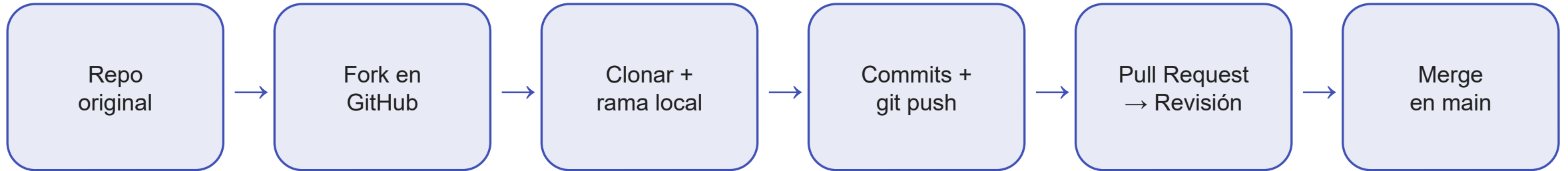
Es el mecanismo estándar en la industria: tanto para contribuir a proyectos de terceros (desde un fork) como para trabajar en equipo dentro del mismo repositorio (desde una rama propia hacia main).

¿Cuándo se usa?

- Contribuir a un proyecto ajeno donde no tienes permisos directos (vía fork).
- Trabajar en equipo en el mismo repositorio: cada desarrollador trabaja en su rama y pide revisión antes de tocar main.
- Documentar y revisar los cambios antes de que lleguen a la rama principal.

Pull Requests — Flujo completo

Del fork al merge



Si el revisor pide cambios: se añaden commits en la misma rama y se hace push. La PR se actualiza automáticamente; no hace falta abrir una nueva.

El proceso puede tener varias rondas: el autor corrige, el revisor revisa de nuevo... hasta que se aprueba o se decide cerrar la PR sin fusionar.

Abrir una Pull Request — Selectores

Cuando subes una rama nueva con git push, GitHub muestra el botón «Compare & pull request». Al pulsarlo llegas al formulario donde configuras cuatro selectores:

Selector	Qué indica	Ejemplo
Base repository	Dónde va a ir el código	torvalds/linux
Base	La rama de destino	main
Head repository	Desde dónde ofreces los cambios	tu-usuario/linux
Compare	La rama con tus commits	arreglo-bug-red

Si la PR es dentro del mismo repositorio (sin fork), base y head son el mismo repositorio y solo cambian las ramas. Además de los selectores, escribe un título claro y una descripción que explique qué problema resuelve el cambio.

Pull Requests — Aviso de nueva rama

Aviso que aparece en el repositorio justo después de hacer push de una rama nueva, con el botón «Compare & pull request» para crear la PR directamente.

 **mi_proyecto** Public

 Pin  Watch 0

 **nueva-rama** had recent pushes 5 seconds ago [Compare & pull request](#)

 **main**  **2 Branches**  **0 Tags** t [Add file](#) [Code](#)

 **aitorventura** Cambios

9e539b6 · yesterday  **2 Commits**

 primer_doc.txt Cambios yesterday

Pull Requests — Formulario de apertura

Formulario de apertura de la PR con los cuatro selectores de rama y el campo de descripción.

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

 base: main  compare: nueva-rama

✓ **Able to merge.** These branches can be automatically merged.

 **Add a title**

Add a description

Write Preview

H B I                                                   

Add your description here...

Markdown is supported  Paste, drop, or click to add files

Create pull request

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

Pull Requests — PR abierta para revisión

La PR ya abierta, donde el equipo puede revisar el código línea a línea, dejar comentarios y aprobar los cambios.

Nueva rama #1

Edit <> Code

Open aitorventura wants to merge 2 commits into main from nueva-rama

Conversation 0 Commits 2 Checks 0 Files changed 2

aitorventura commented now

Nuevo doc

aitorventura added 2 commits yesterday

rama 3a6863a

Nuevo doc 4c598c7

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Pull Requests — Confirmar Merge

El botón «Merge pull request» que activa el administrador una vez el equipo ha dado el visto bueno.



Commit message

Merge pull request #1 from aitorventura/nueva-rama

Extended description

Nueva rama

This commit will be authored by 36103398+aitorventura@users.noreply.github.com.

Confirm merge

Cancel

Still in progress? [Convert to draft](#)

Pull Requests — Fusión completada

La PR en estado «Merged» (etiqueta morada), confirmando que los cambios han pasado a la rama principal.

 **mi_proyecto** Public

 Pin  Watch **0**  Fork **0**  Star **0**

 **main**  **3 Branches**  **0 Tags**

 Add file  **Code**

 **aitorventura** Merge pull request [#1](#) from aitorventura/nueva-rama  d67b0e1 · now  **5 Commits**

 nuevo_archivo.txt	rama	yesterday
 nuevo_doc.txt	Nuevo doc	3 minutes ago
 primer_doc.txt	Cambios	yesterday

About 
No description, website, or topics provided.
 Activity
 **0 stars**
 **0 watching**
 **0 forks**

Releases

Pull Requests — Detalles importantes

PR desde el mismo repo vs desde un Fork

Si la PR viene de una rama del mismo repositorio, los selectores «Base repository» y «Head repository» son iguales. Si contribuyes desde un fork, apuntan a repositorios distintos: el del autor original y el tuyo respectivamente.

Cerrar Issues automáticamente

Si en la descripción de tu PR escribes Closes #10, GitHub cierra automáticamente el Issue #10 en el momento en que se aprueba el merge. Esto evita trabajo de gestión manual.

```
Closes #10    ← cierra el Issue #10 al fusionar
Fixes #10    ← equivalente
Resolves #10 ← equivalente
```

Queda así un historial claro de qué PR resolvió qué problema.

Gestión del proyecto: Issues y Projects

Además del código, GitHub ofrece herramientas integradas para organizar el trabajo del equipo.

	Issues	Projects
Para qué sirve	Registrar tareas, errores o propuestas de mejora	Visualizar y priorizar el trabajo del equipo
Unidad de trabajo	Un ticket por tarea o bug, con su hilo de conversación	Tablero con columnas: To Do, In Progress, Done
Integración	Se cierran automáticamente con Closes #N en PRs	Las tarjetas enlazan a Issues y PRs; se mueven solas al fusionar
Cuándo usarlo	Siempre que hay algo por hacer o un bug que reportar	Cuando el equipo necesita ver el estado global del proyecto

Cada Issue puede tener: responsables (Assignees), etiquetas de colores (Labels: bug, enhancement, documentation...) y fechas de entrega (Milestones).

Otras herramientas del repositorio

Wiki

Espacio de documentación con estructura de páginas. Ideal para manuales de uso, guías para nuevos colaboradores o FAQs. Mantiene el README.md limpio y navegable.

GitHub Actions

Permite automatizar tareas que se ejecutan en respuesta a eventos del repositorio:

- Pasar tests automáticamente cuando alguien abre una PR.
- Desplegar la aplicación cuando se fusiona en main.
- Generar documentación automáticamente tras cada push.

GitHub Actions se verá en detalle en la sección 9: Integración continua con GitHub Actions.

Protecciones de ramas

Cuando varias personas hacen PRs hacia main, conviene blindar esa rama para evitar que alguien suba código directamente sin revisión. En Settings → Rules los administradores configuran las reglas.

Reglas que se pueden activar

- Todo el código debe entrar por PR — el push directo a main queda bloqueado.
- Al menos un compañero tiene que revisar y aprobar el código antes del merge.
- Los tests automatizados deben pasar antes de que el botón de merge se active.

Esto no es burocracia: es lo que impide que un error de un miembro del equipo rompa el proyecto para todos los demás.

Ideas clave

- Fork: copia de un repositorio ajeno bajo tu cuenta. Tú tienes control total; el original no se toca.
- Sync fork: sincroniza tu fork con los nuevos commits del repositorio original (upstream).
- Pull Request: propuesta formal para integrar una rama en otra. Pasa por revisión del equipo antes del merge.
- Closes #10 en la descripción de una PR cierra el Issue #10 automáticamente al hacer merge.
- Issues: sistema de seguimiento de tareas y errores. Cada tarea es un ticket numerado.
- Projects: tablero visual que agrupa Issues y PRs por estado (To Do / In Progress / Done).
- Branch protection: reglas que obligan a usar PRs y pasar tests antes de tocar main.